RCS-AC 알고리즘 상세 계획서

Risk-aware Counterfactual Shapley Actor-Critic

악천후 및 센서 드랍 환경을 고려한
멀티센서 자율주행용 Shapley-inspired 강화학습 방법론

문서 범위
본 문서는 제안 알고리즘 RCS-AC의 **1장부터 12장까지**를 정리한 구현 지향형 설계 문서이다. 포함 범위는 알고리즘 개요, 입력/출력, 전체 구조, 세부 모듈, 손실 함수, 학습 환경 구성, 단계별 학습 절차, Shapley 근사 계획, 보상 구조, 그리고 baseline 대비 핵심 차별점까지이다.

문서 성격	구현 및 실험 설계를 위한 정리본
핵심 키워드	Shapley-inspired fusion, reliability estimation, risk-aware gating, dropout consistency, recurrent actor-critic
적용 환경	CARLA 기반 RGB / LiDAR / ego-state / route 멀티모달 자율주행

Contents

1 알고리즘 개요 **4**
1.1 알고리즘 이름 . . . 4
1.2 핵심 목표 . . . 4

2 왜 이 알고리즘을 쓰는가 **4**

3 입력과 출력 정의 **5**
3.1 입력 모달 . . . 5
3.1.1 (1) RGB 카메라 . . . 5
3.1.2 (2) LiDAR . . . 5
3.1.3 (3) ego-state . . . 5
3.1.4 (4) route . . . 5
3.2 출력 action . . . 6

4 전체 구조 **6**

5 모듈별 상세 설계 **6**
5.1 RGB Encoder . . . 7
5.1.1 역할 . . . 7
5.1.2 제안 구조 . . . 7
5.1.3 출력 . . . 7
5.2 LiDAR Encoder . . . 7
5.2.1 역할 . . . 7
5.2.2 제안 구조 . . . 7
5.2.3 출력 . . . 7
5.3 Ego Encoder . . . 7
5.3.1 역할 . . . 7
5.3.2 제안 구조 . . . 7
5.3.3 출력 . . . 8
5.4 Route Encoder . . . 8
5.4.1 역할 . . . 8
5.4.2 제안 구조 . . . 8
5.4.3 출력 . . . 8

5.5 Reliability Head 8
5.5.1 역할 8
5.5.2 입력 8
5.5.3 구조와 출력 8
5.6 Risk / Context Head 9
5.6.1 역할 9
5.6.2 입력 예시 9
5.6.3 출력 9
5.7 Counterfactual Contribution Estimator 9
5.7.1 목표 9
5.7.2 훈련 시 구현 방식 10
5.7.3 출력 정규화 10
5.8 Fusion Gate 10
5.8.1 역할 10
5.8.2 안전 바닥 가중치 10
5.9 Fusion Output 11
5.10 Recurrent Backbone 11
5.10.1 역할 11
5.10.2 구조 11
5.11 Actor Head 11
5.12 Critic Head 11

6 손실 함수 설계 12
6.1 Actor Loss 12
6.2 Value Loss 12
6.3 Entropy Loss 12
6.4 Shapley Alignment Loss 12
6.5 Reliability Supervision Loss 13
6.6 Dropout Consistency Loss 13
6.7 Risk-aware Regularization 13

7 학습 데이터 및 환경 구성 13
7.1 정상 환경 13
7.2 악천후 환경 13

7.2.1 Level 1 . . . 14
7.2.2 Level 2 . . . 14
7.2.3 Level 3 . . . 14
7.3 센서 드랍 및 열화 . . . 14
7.3.1 RGB . . . 14
7.3.2 LiDAR . . . 14
7.3.3 Ego . . . 14
7.3.4 Route . . . 14

8 학습 단계 계획 14
8.1 Stage 0. 환경 검증 . . . 15
8.2 Stage 1. Warm-up . . . 15
8.3 Stage 2. Mild robustness learning . . . 15
8.4 Stage 3. Full RCS-AC training . . . 16

9 Rollout 및 Update 계획 16

10 Shapley 근사 계산 계획 16
10.1 훈련 중 . . . 16
10.2 평가 시 . . . 17

11 보상 함수 권장 구조 17

12 RCS-AC가 Baseline보다 달라지는 핵심 17
12.1 PPO + attention 대비 차이 . . . 18
12.2 SAC/TD3 + attention 대비 차이 . . . 18

1. 알고리즘 개요

1.1 알고리즘 이름

RCS-AC (Risk-aware Counterfactual Shapley Actor-Critic)

1.2 핵심 목표

RCS-AC는 RGB, LiDAR, ego-state, route의 네 개 모달을 함께 사용하면서, 다음 세 요소를 동시에 반영하도록 설계된 멀티모달 자율주행 강화학습 방법이다.

- 현재 주행 상황의 **위험도(risk)**
- 각 센서가 현재 얼마나 믿을 만한지에 대한 **신뢰도(reliability)**
- 각 센서가 정책 및 가치 추정에 실제로 얼마나 기여하는지에 대한 **기여도(contribution)**

이 알고리즘의 목적은 정상 환경에서의 절대 성능만 높이는 것이 아니라, **악천후와 센서 드랍 상황에서 성능 감소폭을 줄이는 강건한 정책**을 학습하는 데 있다.

> **요약**
> RCS-AC는 "중요해 보이는 모달"만 보는 구조가 아니라, "지금 믿을 수 있는 모달"과 "실제로 가치 추정에 도움이 된 모달"을 함께 고려하여 최종 fusion weight를 결정하는 구조이다.

2. 왜 이 알고리즘을 쓰는가

기존 attention fusion은 모달별 중요도를 학습할 수 있다는 장점이 있으나, 다음과 같은 한계가 있다.

- RGB가 안개, 저조도, 블러 등으로 품질이 크게 저하된 경우를 직접 반영하기 어렵다.
- LiDAR가 sparsity 증가나 point drop으로 열화된 상황을 명시적으로 처리하지 못할 수 있다.
- 위험한 상황에서 어떤 센서를 더 보수적으로 신뢰해야 하는지에 대한 구조적 제약이 약하다.

RCS-AC는 이러한 문제를 해결하기 위해 아래 네 가지 축을 결합한다.

1. 모달별 encoder 분리
2. 모달별 reliability 추정
3. counterfactual Shapley 근사를 통한 contribution 추정
4. risk-aware fusion과 dropout consistency regularization

핵심 설계 철학

단순히 "무엇이 중요해 보이는가"를 넘어서, "무엇이 실제로 도움이 되었는가"와 "지금 무엇을 믿어도 안전한가"를 함께 반영하도록 설계한다.

3. 입력과 출력 정의

3.1 입력 모달

각 time step t에서 관측 o_t는 다음 네 개 모달로 구성한다.

3.1.1 (1) RGB 카메라

- 전방 이미지
- shape: (C, H, W)
- 제안 해상도: (84×84), (96×96), (128×72)

3.1.2 (2) LiDAR

- point cloud 또는 BEV occupancy grid
- 제안 입력 형태: $(1, H_b, W_b)$ 또는 (K, H_b, W_b)
- 제안 grid 크기: (64×64), (96×96), (128×128)

3.1.3 (3) ego-state

예시 입력은 다음과 같다.

- speed
- yaw rate
- acceleration
- steer
- heading error
- lane offset
- 최근 control change

차원 예시: $(6 \sim 12)$

3.1.4 (4) route

예시 입력은 다음과 같다.

- next waypoint distance
- next waypoint heading difference
- lookahead waypoint $(3 \sim 8)$개 요약값
- target distance
- curvature proxy

차원 예시: $(8 \sim 24)$

3.2 출력 action

연속 제어를 사용하며 action은 다음과 같다.

- steering: $[-1, 1]$
- throttle: $[0, 1]$
- brake: $[0, 1]$

출력 분포는 다음 중 하나를 선택한다.

- **권장안 A**: steering은 Gaussian, throttle과 brake는 Beta 또는 sigmoid Gaussian squashing
- **권장안 B**: 세 action 모두 squashed Gaussian

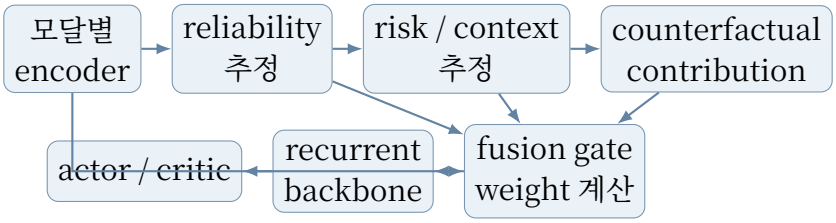
초기 구현은 일반적으로 **권장안 B**가 더 단순하다.

4. 전체 구조

전체 입력은 다음과 같이 정의한다.

$$o_t = \{o_t^{rgb}, o_t^{lidar}, o_t^{ego}, o_t^{route}\}.$$

전체 처리 흐름은 아래와 같다.



보다 구체적으로는 다음 순서를 따른다.

1. 각 모달별 encoder로 feature 추출
2. 각 모달별 reliability score 추정
3. 위험도 및 주행 문맥(context) 추정
4. counterfactual critic로 contribution 추정
5. reliability, contribution, risk를 사용해 fusion weight 계산
6. fused feature를 recurrent backbone에 입력
7. actor와 critic이 action 및 value 출력
8. 기본 RL loss와 auxiliary loss를 함께 사용하여 공동 학습

5. 모듈별 상세 설계

5.1 RGB Encoder

5.1.1 역할

이미지에서 차선, 전방 구조, 객체, 시야 상태를 추출한다.

5.1.2 제안 구조

- CNN 또는 작은 ResNet
- convolution block 수: $(3 \sim 5)$
- channel 규모: $(32 \sim 256)$
- 최종 feature dimension: $(128 \sim 512)$

5.1.3 출력

$$h_t^{rgb} \in \mathbb{R}^{d_h}$$

여기서 d_h의 제안값은 $(128 \sim 256)$이다.

5.2 LiDAR Encoder

5.2.1 역할

거리 구조, 장애물 밀도, 전방 점유 상태를 추출한다.

5.2.2 제안 구조

- 2D CNN 기반 BEV encoder
- convolution block 수: $(3 \sim 5)$
- 채널 수: $(32 \sim 256)$
- 최종 feature dimension: $(128 \sim 512)$

5.2.3 출력

$$h_t^{lidar} \in \mathbb{R}^{d_h}$$

RGB encoder와 동일한 hidden dimension으로 projection하는 구성을 권장한다.

5.3 Ego Encoder

5.3.1 역할

차량 내부 상태와 동역학 정보를 벡터 형태로 표현한다.

5.3.2 제안 구조

- MLP layer 수: $(2 \sim 4)$
- hidden size: $(64 \sim 256)$

5.3.3 출력

$$h_t^{ego} \in \mathbb{R}^{d_h}$$

5.4 Route Encoder

5.4.1 역할

향후 이동 방향, 곡률, 목표 접근 정보를 표현한다.

5.4.2 제안 구조

- MLP layer 수: $(2 \sim 4)$
- hidden size: $(64 \sim 256)$

5.4.3 출력

$$h_t^{route} \in \mathbb{R}^{d_h}$$

5.5 Reliability Head

5.5.1 역할

각 모달이 현재 얼마나 신뢰할 만한지를 점수화한다. 각 모달 m에 대해,

$$q_t^m \in [0, 1]$$

를 출력한다.

5.5.2 입력

각 reliability head는 다음 두 종류 정보를 함께 사용한다.

- **모달 feature**: h_t^m
- **품질 관련 통계**

예시는 다음과 같다.

- **RGB**: 평균 밝기, contrast, blur proxy, frame dropout flag, rain/fog severity, image entropy
- **LiDAR**: point count, valid point ratio, near/far density, drop ratio, sparsity metric
- **Ego**: sensor noise flag, jerk abnormality, recent state inconsistency
- **Route**: path availability flag, waypoint continuity, heading mismatch

5.5.3 구조와 출력

- modality-specific MLP
- layer 수: $(2 \sim 3)$
- hidden size: $(32 \sim 128)$

출력은 다음과 같다.

$$q_t^{rgb},\ q_t^{lidar},\ q_t^{ego},\ q_t^{route}$$

5.6 Risk / Context Head

5.6.1 역할

현재 주행 문맥과 위험도를 요약한다.

5.6.2 입력 예시

- speed
- lane offset
- heading error
- curve proxy
- target distance
- recent collision near-miss proxy
- weather severity
- sensor dropout mask
- LiDAR density
- RGB visibility proxy

5.6.3 출력

$$c_t \in \mathbb{R}^{d_c}, \qquad r_t \in [0,1]$$

여기서 context embedding 차원의 제안 범위는 $(32 \sim 128)$이다.

5.7 Counterfactual Contribution Estimator

5.7.1 목표

각 모달이 가치 추정에 실제로 얼마나 도움을 주었는지를 근사한다.

모달 m에 대한 contribution score ϕ_t^m는 다음과 같이 정의한다.

$$\phi_t^m \approx \frac{1}{K} \sum_{S \subseteq M \setminus \{m\}} \left[V(o_t^{S \cup \{m\}}, \mu_{S \cup \{m\}}) - V(o_t^S, \mu_S) \right]$$

여기서,

- $M = \{rgb, lidar, ego, route\}$
- S: 모달 subset
- μ_S: subset mask
- V: masked critic value

- K: 샘플링하는 subset 수, 제안 범위 $(3 \sim 8)$

5.7.2 훈련 시 구현 방식

훈련 중에는 exact Shapley 대신 approximate Shapley를 사용한다.

- 각 update마다 일부 batch만 선택
- 각 sample마다 subset $(3 \sim 8)$개 랜덤 선택
- mask token 또는 zero masking 사용
- critic이 "이 모달 조합일 때의 value"를 예측

5.7.3 출력 정규화

$$\hat{c}_t^m = \text{softmax}\left(\frac{\phi_t^m}{\tau_c}\right)$$

온도 파라미터 τ_c의 제안 범위는 $(0.3 \sim 1.5)$이다.

5.8 Fusion Gate

5.8.1 역할

실제 fusion weight를 계산한다.

각 모달 m에 대해 gate logit은 다음과 같다.

$$g_t^m = f_g\left(h_t^m,\ q_t^m,\ \hat{c}_t^m,\ r_t,\ c_t\right)$$

최종 weight는 다음과 같이 계산한다.

$$w_t^m = \text{softmax}\left(\frac{g_t^m}{\tau_w}\right)$$

온도 파라미터 τ_w의 제안 범위는 $(0.5 \sim 2.0)$이다.

5.8.2 안전 바닥 가중치

자율주행에서는 ego와 route가 완전히 무시되면 위험할 수 있으므로, 최소 가중치 바닥을 두는 구성을 권장한다.

$$w_t^{route} \geq \epsilon_{route}, \qquad w_t^{ego} \geq \epsilon_{ego}$$

제안 범위는 다음과 같다.

- $\epsilon_{route} = (0.05 \sim 0.20)$
- $\epsilon_{ego} = (0.03 \sim 0.15)$

바닥 적용 후에는 다시 정규화한다.

5.9 Fusion Output

모달별 projection 후 weighted sum을 사용한다.

$$z_t = \sum_{m \in M} w_t^m P_m(h_t^m)$$

여기서 P_m은 projection layer이다.

출력 dimension은 다음과 같다.

$$z_t \in \mathbb{R}^{d_z}$$

d_z의 제안 범위는 $(128 \sim 512)$이다.

5.10 Recurrent Backbone

5.10.1 역할

부분관측성과 일시적 센서 손상에 대응한다.

5.10.2 구조

- GRU 또는 LSTM
- layer 수: $(1 \sim 2)$
- hidden size: $(128 \sim 512)$

상태 업데이트는 다음과 같다.

$$u_t = \mathrm{RNN}(z_t, u_{t-1})$$

5.11 Actor Head

입력은 u_t이며, 출력은 행동 분포 파라미터이다. 예시는 다음과 같다.

- steering mean / std
- throttle mean / std
- brake mean / std

5.12 Critic Head

기본 critic은 다음과 같다.

$$V_t = V(u_t)$$

Shapley 계산용 masked critic은 다음과 같이 별도로 둔다.

$$V_t^{mask} = V_{mask}(u_t, \mu_t)$$

권장 구현은 다음 두 가지 중 하나이다.

- shared trunk + separate value head

- shared recurrent trunk 이후 mask-conditioned critic head

6. 손실 함수 설계

최종 손실은 다음과 같이 구성한다.

$$L_{total} = L_{actor} + \lambda_v L_{value} + \lambda_e L_{entropy} + \lambda_s L_{shap} + \lambda_q L_{rel} + \lambda_c L_{cons} + \lambda_r L_{risk}$$

각 λ는 고정 상수가 아니라 개발 과정에서 조정 가능한 **제안 범위**로 둔다.

6.1 Actor Loss

안정성을 위해 PPO-style clipped actor loss를 권장한다.

$$L_{actor} = -\mathbb{E}\left[\min\left(\rho_t A_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)A_t\right)\right]$$

clip 파라미터 ϵ의 제안 범위는 $(0.1 \sim 0.3)$이다.

6.2 Value Loss

$$L_{value} = \mathbb{E}\left[(V_t - R_t)^2\right]$$

Huber loss로 대체 가능하며, 계수 λ_v의 제안 범위는 $(0.25 \sim 1.0)$이다.

6.3 Entropy Loss

탐험 유지용 손실은 다음과 같다.

$$L_{entropy} = -\mathbb{E}[H(\pi(\cdot \mid u_t))]$$

계수 λ_e의 제안 범위는 $(0.001 \sim 0.05)$이다.

6.4 Shapley Alignment Loss

Fusion gate의 weight가 counterfactual contribution과 지나치게 다르지 않도록 정렬한다.

$$L_{shap} = \mathbb{E}\left[\sum_m (w_t^m - \hat{c}_t^m)^2\right]$$

대안으로 KL divergence를 사용할 수도 있다.

$$L_{shap}^{KL} = \mathbb{E}\left[D_{KL}(\hat{c}_t \parallel w_t)\right]$$

계수 λ_s의 제안 범위는 $(0.05 \sim 1.0)$이다.

6.5 Reliability Supervision Loss

훈련 중 주입한 corruption 정보를 사용해 pseudo-label $\tilde{q}_t^m$를 만들고, 각 모달의 reliability와 맞춘다.

$$L_{rel} = \mathbb{E}\left[\sum_m (q_t^m - \tilde{q}_t^m)^2\right]$$

계수 λ_q의 제안 범위는 $(0.05 \sim 0.5)$이다.

6.6 Dropout Consistency Loss

full input과 dropout input 사이에서 정책이 과도하게 흔들리지 않도록 제약한다.

$$L_{cons} = \mathbb{E}\left[D_{KL}(\pi_{full} \parallel \pi_{drop})\right]$$

또는 다음과 같이 action 차이를 직접 줄일 수도 있다.

$$L_{cons} = \mathbb{E}\left[\|a_{full} - a_{drop}\|^2\right]$$

계수 λ_c의 제안 범위는 $(0.05 \sim 1.0)$이다.

6.7 Risk-aware Regularization

위험한 상황일수록 신뢰도 낮은 센서에 대한 과의존을 줄이도록 한다.

$$L_{risk} = \mathbb{E}\left[r_t \sum_m w_t^m (1 - q_t^m)\right]$$

모달별 penalty를 두는 확장형은 다음과 같다.

$$L_{risk} = \mathbb{E}\left[r_t \sum_m \alpha_m w_t^m (1 - q_t^m)\right]$$

여기서 α_m의 제안 범위는 $(0.5 \sim 2.0)$이고, 계수 λ_r의 제안 범위는 $(0.05 \sim 1.0)$이다.

7. 학습 데이터 및 환경 구성

7.1 정상 환경

- clear weather (맑음)
- daytime (낮)
- low traffic (낮은 교통량)
- no sensor dropout (없음)

7.2 악천후 환경

악천후는 강도를 세 단계로 나누는 것을 권장한다.

7.2.1 Level 1

- light rain (약한 비)
- light fog (약한 안개)
- slight low-light (약한 저조도)

7.2.2 Level 2

- moderate rain (중간 비)
- moderate fog (중간 안개)
- dusk/night (해질녘/야간)

7.2.3 Level 3

- heavy rain (강한 비)
- dense fog (짙은 안개)
- strong low-light (강한 저조도)

7.3 센서 드랍 및 열화

각 에피소드 시작 시점 또는 time step 단위로 랜덤 적용 가능하도록 설계한다.

7.3.1 RGB

- frame dropout rate: $(0.05 \sim 0.40)$
- blur severity: (낮음 ~ 높음)
- resolution downscale factor: $(1/1.5 \sim 1/4)$

7.3.2 LiDAR

- point drop rate: $(0.05 \sim 0.60)$
- range attenuation severity: (낮음 ~ 높음)
- sparsity increase: (낮음 ~ 높음)

7.3.3 Ego

- Gaussian noise scale: (작음 ~ 중간)

7.3.4 Route

- lookahead corruption probability: $(0.0 \sim 0.2)$

8. 학습 단계 계획

8.1 Stage 0. 환경 검증

- random 또는 simple heuristic policy로 환경 안정성 확인
- reward, done, sensor synchronization 확인

8.2 Stage 1. Warm-up

정상 환경만 사용하고 auxiliary loss는 최소화하거나 일부 비활성화한다.

활성화

- L_{actor}
- L_{value}
- $L_{entropy}$

약하게 활성화

- L_{rel}

비활성화 또는 매우 낮게 시작

- L_{shap}
- L_{cons}
- L_{risk}

목표

- 기본 주행이 되는지 확인
- fusion 구조가 비정상적이지 않은지 확인
- learning curve가 상승하는지 확인

8.3 Stage 2. Mild robustness learning

약한 악천후와 약한 dropout을 추가한다.

활성화

- L_{actor}
- L_{value}
- $L_{entropy}$
- L_{rel}
- L_{cons}

낮게 활성화

- L_{shap}
- L_{risk}

목표

- reliability head가 의미 있는 값을 내는지 확인
- full input과 corrupted input 사이의 consistency 형성 여부 확인

8.4 Stage 3. Full RCS-AC training

중간 강한 악천후와 센서 드랍을 포함하여 전체 학습을 수행한다.

모든 loss 활성화

- L_{actor}
- L_{value}
- $L_{entropy}$
- L_{rel}
- L_{shap}
- L_{cons}
- L_{risk}

목표

- 악조건에서 덜 무너지는 정책 확보
- contribution-aware fusion 안정화

9. Rollout 및 Update 계획

학습 스케줄 관련 제안 범위는 다음과 같다.

항목	제안 범위
rollout length	$(128 \sim 2048)$ steps / update
mini-batch 수	$(4 \sim 16)$
epoch 수	$(2 \sim 10)$
learning rate (actor)	$(1e-5 \sim 5e-4)$
learning rate (critic)	$(1e-5 \sim 5e-4)$
learning rate (auxiliary heads)	$(1e-5 \sim 5e-4)$
gradient clipping	max grad norm $(0.3 \sim 1.5)$
discount factor	$\gamma = (0.95 \sim 0.999)$
GAE lambda	$(0.90 \sim 0.99)$

10. Shapley 근사 계산 계획

10.1 훈련 중

모든 step에서 exact Shapley를 계산하지 않고, 주기적으로 일부 샘플에 대해서만 수행한다.

제안 방식

- every $(1 \sim 8)$ updates마다 수행
- batch의 일부 샘플 $(10\% \sim 50\%)$만 대상으로 선택
- subset은 $(3 \sim 8)$개만 샘플링

이 방식은 계산량을 줄이면서도 contribution estimator에 필요한 감독 신호를 유지하기 위한 절충안이다.

10.2 평가 시

모달 수가 4개이므로, 평가 단계에서는 exact에 가까운 계산이 가능하다.

가능한 방식

- 모달 하나당 가능한 subset 조합을 모두 계산
- 고정된 scenario에서 contribution map을 추출
- 논문 및 발표용 해석 자료로 활용

11. 보상 함수 권장 구조

보상 함수는 지나치게 복잡하게 만들지 않고, 주행 목적과 안정성 항을 분리한 형태를 권장한다.

$$r_t = w_p r_{progress} + w_l r_{lane} + w_g r_{goal} - w_c r_{collision} - w_o r_{offroad} - w_s r_{smooth} - w_b r_{stuck}$$

각 계수는 고정 상수로 박지 않고 다음과 같은 제안 범위로 관리한다.

계수	제안 범위
w_p	$(0.2 \sim 2.0)$
w_l	$(0.05 \sim 0.5)$
w_g	$(2.0 \sim 20.0)$
w_c	$(3.0 \sim 20.0)$
w_o	$(1.0 \sim 10.0)$
w_s	$(0.01 \sim 0.2)$
w_b	$(0.5 \sim 5.0)$

보상 설계 원칙

주행 progress를 유도하는 항과 위험 회피 항의 균형이 중요하다. goal 보상과 collision/offroad penalty가 지나치게 크면 정책이 소극적으로 수렴할 수 있고, 반대로 progress 항이 너무 강하면 과속 또는 offroad 성향이 생길 수 있다.

12. RCS-AC가 Baseline보다 달라지는 핵심

12.1 PPO + attention 대비 차이

PPO + attention은 기본적으로 learned importance에 의해 모달 weight를 조절한다. 반면 RCS-AC는 learned importance에만 의존하지 않고, 다음 세 요소를 함께 사용한다.

- reliability
- counterfactual contribution
- risk-aware weighting

즉, attention이 "중요해 보이는 모달"을 학습한다면, RCS-AC는 "중요하면서도 믿을 수 있고, 현재 상황에서 안전한 모달"을 우선시한다.

12.2 SAC/TD3 + attention 대비 차이

SAC 또는 TD3에 attention을 붙인 baseline은 RL update rule만 다르고 fusion 철학은 유사할 수 있다. RCS-AC는 update rule보다도 **fusion 방식 자체**를 바꾼다는 점에서 차별화된다.

> **정리**
> RCS-AC의 핵심 차별점은 단순한 중요도 학습을 넘어서, **기여도(contribution)**, **신뢰도(reliability)**, **위험도(risk)**를 동시에 반영하는 동적 fusion에 있다. 따라서 정상 환경에서의 절대 성능보다, 악천후 및 센서 드랍 환경에서의 **성능 감소폭 감소**가 더 중요한 평가 포인트가 된다.

본 문서는 요청 범위에 따라 1장부터 12장까지의 내용을 정리한 PDF 버전이다.